

contains a call to the function 'bar', then the compiler examines the code of both 'foo' and 'bar' and in certain cases, decides to inline the function 'bar' at the call site in 'foo'. Inlining is a complex transformation, since indiscriminate inlining can cause considerable code bloat. Can you think of the reasons why the compiler performs inlining? It has two benefits. First, it reduces the function call overheads. Performing a call involves certain overheads, such as passing the parameters, storing the return address and saving registers. Inlining avoids these overheads. Second, inlining increases the code that is visible to the compiler for performing optimisations. This is what makes inlining a powerful tool for improving the performance of a given application.

Remember that the compiler performs most of the classical optimisations we have discussed in the previous column, at a function level. Therefore, inlining allows the compiler to examine a larger region of code for potential optimisation opportunities. For example, consider the following piece of code:

```
Int A[N];
Foo()
{
    ...
    for (int i = 0; i < N; i++)
        = A[i];
    bar(N);
    ...
}
bar(int K)
{
    For (int j = 0; j < K; j++)
        = A[j];
    ...
}
```

In the above code snippet, we have a for loop in function 'foo', the loop index of which has the same iteration space as the for loop in the function 'bar'. The function 'bar' is invoked right after the 'for' loop in function

'foo'. Both these 'for' loops iterate over the array A. Therefore, it would be beneficial if these two loops can be fused by the compiler by performing the loop fusion transformation. However, these two 'for' loops occur in two different functions. Almost all the classical scalar and loop transformations happen within a single function unit. Hence, it is not possible for the compiler to apply loop fusion to these two 'for' loops, since they are in different functions. However, once inter-procedural analysis and transformation has been done by the compiler, the compiler can inline the function 'bar' into the function 'foo'. The compiler performs loop nest optimisations downstream to the inlining phase. As part of this loop nest optimisation phase on function 'foo', the compiler can now apply standard loop fusion transformation to these two 'for' loops. Thus, inlining enables downstream optimisations, thereby considerably aiding performance.

While inlining is a well-known inter-procedural transformation, there are a number of lesser-known but quite powerful inter-procedural transformations. One of them is known as inter-procedural constant propagation. Most of us would be familiar with local constant propagation. Consider the following piece of code:

```
Foo()
{
    ..... /* do some thing */
    Int I = 20;
    Result = I * 10;
    Return Result;
}
```

In the above code snippet, the compiler can perform constant propagation and return the result as 200. However, consider the following code snippet:

```
Foo (int I)
{
    ..... /* do some thing which does
```

RHCVA

(Red Hat Certified Virtualization Administrator)

The future is of Virtualization

@IPSR

Get trained by faculty team headed by **RHCA** Certified Professionals



redhat TRAINING PARTNER

Produced **42** **RHCSS** Highest in Asia Pacific

Special OFFER for RHCSS RHCVA

RHCA RHCDS RHCSS RHCVA RHCE

Modules Starting Every Week

Awards

- Winner of Red Hat Asia Pacific International Award 2008, 2009 & 2010
- Winner of Red Hat GLS National Awards 2004, 05, 06, 07 & 08

Faculty & Infrastructure

- 15 member faculty crew led by RHCE, RHCVA, RHCSS, RHCDS, RHCA certified professionals
- Global standard infrastructure offered at our International Centre located in Kochi

Achievements

- Produced 3000+ RHCEs, 40 RHCSS & 3 RHCA

ipshr solutions ltd
red*ining excellence

Kochi South • Kochi North • Kozhikode
Trivandrum • Kottayam
Tel : 0484 - 2344560, 0481 - 2561410 / 20
Mob: 09447294635, 09447169776
www.ipshr.org • training@ipsrsolutions.com

IPSR, The Best in Red Hat Certifications